

# CYBERGUARD PASSWORD ANALYZER

Evaluated Course Project — Python Essentials

---

|                         |                        |
|-------------------------|------------------------|
| <b>Student Name</b>     | Devanshi Kabra         |
| <b>Registration No.</b> | 26BCY10085             |
| <b>Institution</b>      | VIT Bhopal University  |
| <b>Course</b>           | Python Essentials      |
| <b>Faculty</b>          | Mr Devendra Kumar Vedi |
| <b>Academic Year</b>    | 2026                   |
| <b>Semester</b>         | 1                      |
| <b>Platform</b>         | VITyarthi              |

Project purpose: a modular command-line Python application that gives immediate feedback on password composition, strength, entropy, common-password status, and security recommendations.

Final report prepared from the implemented CyberGuard project and the supplied VITyarthi project requirements. The screenshots in this report are the user's actual command-line execution and automated test results.

# 1. Introduction

CyberGuard Password Analyzer is a command-line Python application developed for the Python Essentials course. It provides local, immediate feedback about a password's basic composition and estimated security characteristics. The application checks password length and character categories, calculates a five-point composition score, classifies password strength and security level, estimates entropy, checks a small built-in set of commonly used passwords, and generates recommendations.

The project is implemented with the Python Standard Library so that it can be executed from the Windows command line without third-party dependency installation.

## 2. Problem Statement

Users may not know whether a password satisfies basic composition requirements or how different password characteristics affect a simple security assessment. CyberGuard addresses this by accepting a password through a terminal interface and producing a structured report containing measurable checks and recommendations.

## 3. Objectives

- Apply Python Essentials concepts to a practical, security-related command-line problem.
- Create a modular Python project instead of keeping all functionality in one file.
- Validate user input and handle edge cases safely.
- Provide understandable password-strength feedback and recommendations.
- Demonstrate automated testing using Python's unittest framework.
- Maintain a clear project structure with documentation suitable for GitHub submission.

## 4. Scope and Target Users

**Scope:** local analysis of a password supplied interactively through the command line. The application does not transmit passwords to an external service and does not require a database.

**Target users:** students and beginner Python learners who want a simple demonstration of password-composition analysis and modular Python programming.

## 5. Functional Requirements

- **FR1 — Password composition analysis:** determine password length and whether uppercase, lowercase, numeric, and special characters are present.
- **FR2 — Security assessment:** calculate a five-point score and classify password strength and security level.
- **FR3 — Entropy and common-password checks:** estimate entropy from detected character categories and compare the input with a small built-in common-password set.
- **FR4 — Recommendations and report:** generate improvement suggestions and display all results in a structured terminal report.
- **FR5 — Repeatable command-line workflow:** allow the user to analyze another password or exit cleanly.

## 6. Non-Functional Requirements

- **Usability:** clear prompts, readable labels, and a structured terminal report.
- **Maintainability:** responsibilities are separated into focused Python modules.
- **Reliability:** invalid and empty password inputs are validated, and entropy calculation handles the empty case safely.

- **Resource efficiency:** analysis is local and uses only the Python Standard Library.
- **Privacy:** the application does not transmit the entered password to a remote service.
- **Error handling:** invalid menu responses are rejected and the user is prompted again.

## 7. Technology and Python Concepts Used

| Area               | Use in CyberGuard                                            |
|--------------------|--------------------------------------------------------------|
| Input / Output     | Terminal prompts and formatted password reports              |
| Strings            | Password inspection and character-category checks            |
| Control Flow       | Conditional classification and repeat-analysis loop          |
| Functions          | Scoring, entropy, recommendations, validation, and reporting |
| Modules / Packages | Separate .py files for focused responsibilities              |
| Data Structures    | Lists/sets and structured result values                      |
| OOP                | PasswordAnalyzer class                                       |
| Dataclass          | PasswordAnalysis result container                            |
| Math               | Entropy calculation using logarithms                         |
| Testing            | unittest test suite with 7 tests                             |

## 8. System Architecture

The system follows a modular command-line architecture. **main.py** acts as the controller. It creates a PasswordAnalyzer, receives user input, requests an analysis, and passes the result to the reporting layer. The analyzer coordinates security, entropy, and recommendation logic.

|                                                           |                                      |                                                 |
|-----------------------------------------------------------|--------------------------------------|-------------------------------------------------|
| <b>User</b><br>Enters password                            | <b>main.py</b><br>CLI controller     | <b>PasswordAnalyzer</b><br>Coordinates analysis |
| <b>security.py</b><br>score, strength,<br>common-password | <b>entropy.py</b><br>entropy + level | <b>recommendations.py</b><br>security advice    |
| <b>report.py</b><br>structured output                     | <b>utils.py</b><br>input validation  | <b>config.py</b><br>constants                   |

## 9. Project Structure

```
CyberGuard/
├── main.py
├── analyzer.py
├── security.py
├── entropy.py
├── recommendations.py
├── report.py
├── utils.py
└── config.py
```

```
■■■ tests/
■   ■■■ test_cyberguard.py
■■■ docs/
■   ■■■ design.md
■   ■■■ project_report.md
■■■ README.md
■■■ statement.md
■■■ requirements.txt
■■■ .gitignore
```

## 10. Workflow

1. Start the application with **python main.py**.
2. Enter a password at the terminal prompt.
3. PasswordAnalyzer coordinates the composition, score, strength, security-level, entropy, common-password, and recommendation checks.
4. report.py displays the results.
5. The user chooses whether to analyze another password or exit.

## 11. Design Decisions and Rationale

- The Standard Library was used to keep setup simple and make command-line execution reproducible.
- The PasswordAnalyzer class provides one clear orchestration point for the analysis workflow.
- The result is grouped into a structured PasswordAnalysis dataclass so reporting is separated from calculation.
- Security, entropy, recommendations, utilities, and reporting are separated into modules to improve maintainability.
- A small automated test suite was added to verify important classifications and edge cases.

## 12. Implementation Details

**Password composition:** the application checks length, uppercase letters, lowercase letters, numbers, and special characters.

**Five-point score:** one point is awarded for each basic composition requirement satisfied, including a minimum length of eight characters.

**Strength/security classification:** score boundaries are handled consistently so that weak, moderate, strong, and very-strong results do not conflict.

**Entropy estimate:** the program estimates entropy from the size of the detected character pool and password length. An empty password is handled safely.

**Common-password check:** the program compares the input against a small built-in set of commonly used examples.

**Recommendations:** missing composition requirements are converted into clear suggestions for improvement.

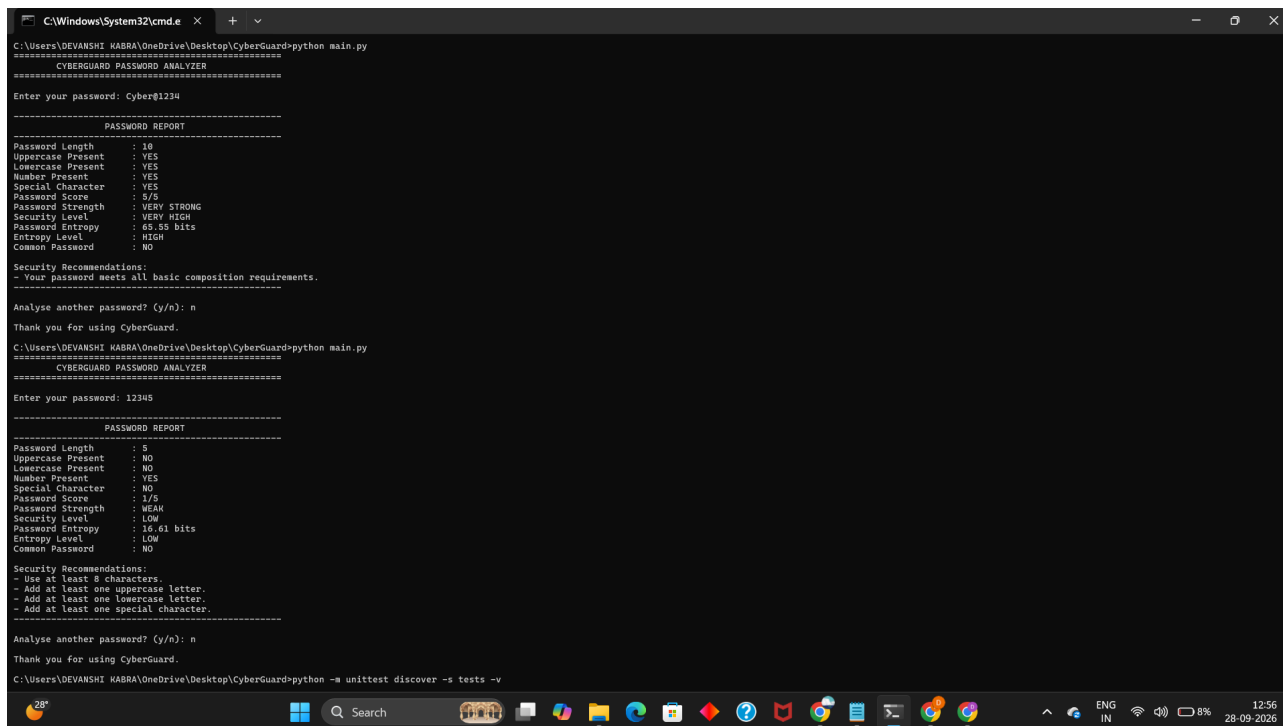
**Validation:** empty or invalid inputs are rejected rather than being processed as normal passwords.

## 13. Screenshots and Results

The following screenshots were supplied from the actual Windows command-line execution of the final project. They document functional output and automated testing.

### 13.1 Strong and Weak Password Analysis

The first screenshot shows two successful analyses. The strong example **Cyber@1234** receives a 5/5 score, VERY STRONG strength, VERY HIGH security level, and HIGH entropy level. The weak example **12345** receives a 1/5 score, WEAK strength, LOW security level, and LOW entropy level.



```
C:\Windows\System32\cmd.exe
C:\Users\DEVANSHI KABRA\OneDrive\Desktop\CyberGuard>python main.py
=====
CYBERGUARD PASSWORD ANALYZER
=====
Enter your password: Cyber@1234

=====
PASSWORD REPORT
=====
Password Length      : 10
Uppercase Present    : YES
Lowercase Present    : YES
Number Present       : YES
Special Character     : YES
Password Score        : 5/5
Password Strength    : VERY STRONG
Security Level        : VERY HIGH
Password Entropy      : 65.55 bits
Entropy Level        : HIGH
Common Password       : NO

Security Recommendations:
- Your password meets all basic composition requirements.

Analyse another password? (y/n): n
Thank you for using CyberGuard.

C:\Users\DEVANSHI KABRA\OneDrive\Desktop\CyberGuard>python main.py
=====
CYBERGUARD PASSWORD ANALYZER
=====
Enter your password: 12345

=====
PASSWORD REPORT
=====
Password Length      : 5
Uppercase Present    : NO
Lowercase Present    : NO
Number Present       : YES
Special Character     : NO
Password Score        : 1/5
Password Strength    : WEAK
Security Level        : LOW
Password Entropy      : 16.61 bits
Entropy Level        : LOW
Common Password       : NO

Security Recommendations:
- Use at least 8 characters.
- Add at least one uppercase letter.
- Add at least one lowercase letter.
- Add at least one special character.

Analyse another password? (y/n): n
Thank you for using CyberGuard.

C:\Users\DEVANSHI KABRA\OneDrive\Desktop\CyberGuard>python -m unittest discover -s tests -v
```

Figure 1. CyberGuard command-line execution showing strong and weak password analysis.

## 13.2 Automated Test Results

The second screenshot shows the automated test command: **python -m unittest discover -s tests -v**. The terminal output reports seven individual tests as passing and ends with **Ran 7 tests** and **OK**.

```
C:\Windows\System32\cmd.exe
Number Present      : YES
Special Character   : YES
Password Score      : 5/5
Password Strength   : VERY STRONG
Security Level      : VERY HIGH
Password Entropy    : 65.55 bits
Entropy Level       : HIGH
Common Password     : NO

Security Recommendations:
- Your password meets all basic composition requirements.

Analyse another password? (y/n): n
Thank you for using CyberGuard.

C:\Users\DEVANSHI KABRA\OneDrive\Desktop\CyberGuard>python main.py
=====
CYBERGUARD PASSWORD ANALYZER
=====

Enter your password: 12345

=====
PASSWORD REPORT
=====
Password Length      : 5
Uppercase Present    : NO
Lowercase Present    : NO
Number Present       : YES
Special Character     : NO
Password Score        : 1/5
Password Strength     : WEAK
Security Level        : LOW
Password Entropy      : 16.61 bits
Entropy Level         : LOW
Common Password       : NO

Security Recommendations:
- Use at least 8 characters.
- Add at least one uppercase letter.
- Add at least one lowercase letter.
- Add at least one special character.

Analyse another password? (y/n): n
Thank you for using CyberGuard.

C:\Users\DEVANSHI KABRA\OneDrive\Desktop\CyberGuard>python -m unittest discover -s tests -v
test_common_password (test_cyberguard.test_cyberguard.test_common_password) ... ok
test_empty_password_rejected (test_cyberguard.test_cyberguard.test_empty_password_rejected) ... ok
test_entropy_empty (test_cyberguard.test_cyberguard.test_entropy_empty) ... ok
test_entropy_level (test_cyberguard.test_cyberguard.test_entropy_level) ... ok
test_score_boundaries (test_cyberguard.test_cyberguard.test_score_boundaries) ... ok
test_strong_password (test_cyberguard.test_cyberguard.test_strong_password) ... ok
test_weak_password (test_cyberguard.test_cyberguard.test_weak_password) ... ok

Ran 7 tests in 0.004s

OK
```

Figure 2. Automated unittest execution showing all 7 tests passing.

## 14. Testing

Testing was performed using Python's built-in unittest framework. The final command-line output supplied for this report shows all seven included tests passing.

| Test                         | Purpose                                             |
|------------------------------|-----------------------------------------------------|
| test_common_password         | Checks common-password detection.                   |
| test_empty_password_rejected | Checks validation of empty input.                   |
| test_entropy_empty           | Checks safe entropy handling for an empty password. |
| test_entropy_level           | Checks entropy-level classification.                |
| test_score_boundaries        | Checks score classification boundaries.             |
| test_strong_password         | Checks a strong password result.                    |
| test_weak_password           | Checks a weak password result.                      |

**Observed result:** 7 tests ran successfully and the final test status was **OK**, as shown in Figure 2.

## 15. Challenges Faced

- Converting the original single-file implementation into a modular project with separate responsibilities.
- Keeping password-strength and security-level classifications consistent at score boundaries.
- Handling empty input and entropy edge cases safely.
- Adding automated tests and arranging the test file under the tests directory.
- Organizing the project for a GitHub submission with README, statement, documentation, tests, and source modules.

## 16. Learnings and Key Takeaways

- Functions and control flow can be combined to build a complete command-line workflow.
- Modules and packages help divide a larger Python program into maintainable components.
- Object-oriented programming can be used for orchestration while functions handle focused calculations.
- Input validation and edge-case handling are important for reliable programs.
- Automated tests provide repeatable evidence that important behaviors continue to work.
- Documentation and project structure are part of software development, not just the code itself.

## 17. Future Enhancements

- Expand the common-password dataset with a larger curated collection.
- Add detection for repeated characters and simple sequential patterns.
- Allow configurable password policies for different use cases.
- Add an optional password-generation feature.
- Provide richer reports while continuing to avoid unnecessary storage or transmission of passwords.

## 18. GitHub Project Contents

The intended repository structure is organized so that the main Python modules are at the repository root, automated tests are under **tests/**, and design/report documentation is under **docs/**. The repository also contains README.md, statement.md, requirements.txt, and .gitignore.

**Repository:** <https://github.com/devanshikabra15-cmyk/CyberGuard-Password-Analyzer>

The README provides project overview, features, technology information, structure, setup/run instructions, testing instructions, workflow, and future enhancements. The statement.md file contains the problem statement, scope, target users, and high-level features.

## 19. Conclusion

CyberGuard demonstrates a complete Python project workflow: identifying a practical problem, designing a modular solution, implementing the solution with Python fundamentals and OOP, validating inputs, testing behavior, documenting the architecture, and preparing the project for GitHub-based evaluation. The supplied execution screenshots show successful functional output and a seven-test automated test run ending in OK.

## 20. References

- VITyarthi — Python Essentials project guidelines and submission requirements supplied for this course project.
- Python Standard Library documentation — built-in modules used by the implementation, including unittest, math, dataclasses, pathlib, and related functionality.

## Appendix A — Command-Line Commands Used

### Run the application:

```
python main.py
```

### Run the automated test suite:

```
python -m unittest discover -s tests -v
```

### Check the installed Git version:

```
git --version
```

These commands are intended to be run from the CyberGuard project directory in Windows Command Prompt.

## Appendix B — Final Project File List

main.py — command-line controller  
analyzer.py — PasswordAnalyzer and structured analysis result  
security.py — score, strength, security level, common-password checks  
entropy.py — entropy estimation and entropy-level classification  
recommendations.py — security recommendations  
report.py — terminal report formatting  
utils.py — input and loop validation  
config.py — project constants  
tests/test\_cyberguard.py — automated tests  
docs/design.md — architecture and design documentation  
docs/project\_report.md — project report source document  
README.md — GitHub project documentation  
statement.md — problem statement and project scope  
requirements.txt — dependency declaration  
.gitignore — ignored generated/local files

## Appendix C — Evidence Included in This Report

- Figure 1 documents successful strong- and weak-password analysis in the Windows command line.
- Figure 2 documents the automated unittest run with all seven tests passing.
- The report uses the student/course details supplied for the final submission.